

8장 함수

함수란?

함수(function): 입력을 받아서 특정한 작업을 수행하여서 결과를 반환하는 블랙 박스(상자)와 같다
특정 작업을 수행하기 위한 명령어들의 모음
서로 구별되는 이름이 있음
함수는 특정한 작업을 수행한다.
입력을 받을 수 있고 결과를 반환할 수 있다.

함수의 장점

코드가 중복되는 것을 막을 수 있음 재사용 가능.
프로그램을 모듈로 나눌 수 있어 개발 과정이 쉬워지고 체계적이 되면서 유지보수가 쉬워짐.



반환형 : 함수가 처리를 종료한 후에 호출한 곳으로 반환하는 데이터의 유형
함수이름 : 식별자 규칙 따름. 기능암시하는 이름이면 좋음. 동사+명사

함수 호출

함수의 이름을 써 주는 것. 호출하기 전까지는 실행되지 않음.
함수가 호출되면 실행하고 있는 코드는 잠시 중단되고 함수 body로 이동하여서 함수 몸체 안의 문장이 순차적으로 실행됨.

인수와 매개변수

인수 : 호출 프로그램에 의하여 함수에 실제로 전달되는 값

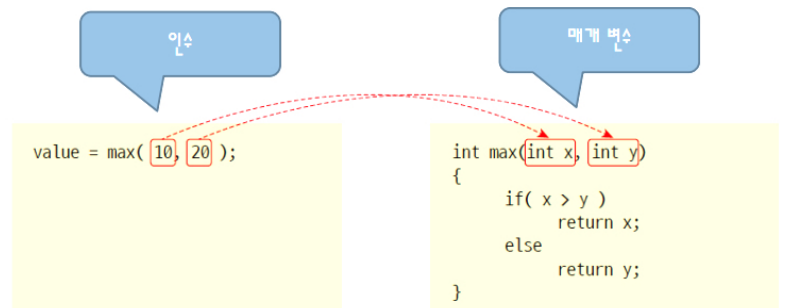
매개 변수 : 이 값을 전달받는 변수

매개변수가 없을때는 void를 쓰거나 빈칸으로
함수가 호출될때마다 인수가 달라질 수 있다.
매개변수의 개수는 정확히 일치해야 함.

반환값

반환값 : 함수가 호출한 곳으로 반환하는 작업의 결과값 반환.
반환값은 하나만 가능!

조합계산 함수



조합 계산하기

```
// 팩토리얼 값을 반환
int factorial(int n)
{
    int i, result = 1;

    for (i = 1; i <= n; i++)
        result *= i;           // result = result * i
    return result;
}

// 팩토리얼 값을 이용하여 조합값을 계산
int combination(int n, int r)
{
    return (factorial(n)/(factorial(r) * factorial(n-r)));
}
```

소수 구하기

```
int is_prime(int n)
{
    int i;

    for (i = 2; i < n; i++) {
        if (n%i == 0)
            return 0;
    }
    return 1;
}
```

함수 원형

main함수 아래에 함수를 선언하면 에러가 난다

```
double c_to_f(double);
int get_integer(void);
```

이렇게 원형을 위에 만들어놔야 함. 세미콜론 필수. 매개변수 이름 필요X

라이브러리 함수

컴파일러에서 제공하는 함수

난수 함수

난수 : 규칙성이 없이 임시로 생성되는 수

rand() : 난수를 생성하는 함수. 0부터 RAND_MAX의 난수를 생성
srand((unsigned)time(NULL)) ; 시간에 따라 시드를 초기화

```
srand( (unsigned)time( NULL ) );

for( i = 0; i < 6; i++ ) {
    car1_dist += rand() % 100;
    car2_dist += rand() % 100;
    disp_car(1, car1_dist);
    disp_car(2, car2_dist);
    printf("-----\n");
}
```

rand()
다. 난수
하여서

math 라이브러리

floor() 내림
ceil() 올림

```
double result, value = 1.6;

result = floor(value);           // result는 1.0이다.
printf("%lf ", result);

result = ceil(value);            // result는 2.0이다.
printf("%lf ", result);
```

fabs() 절댓값

```
printf("12.0의 절대값은 %f\n", fabs(12.0));  
printf("-12.0의 절대값은 %f\n", fabs(-12.0));
```

pow(밑, 제곱)

```
printf("10의 3승은 %.0f\n", pow(10.0, 3.0));  
printf("16의 제곱근은 %.0f\n", sqrt(64));
```

1000

sqrt() 루트

8

```
double pi = 3.1415926535;  
double x, y;
```

cos(double)

```
x = pi / 2;
```

sin(double)

```
y = sin( x );
```

tan(double)

```
printf( "sin( %f ) = %f\n", x, y ); //1.0000000
```

```
y = cos( x );
```

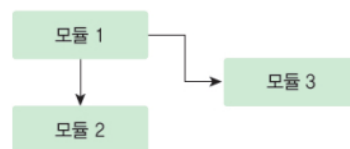
```
printf( "cos( %f ) = %f\n", x, y ); //0.0000000
```

라디안으로 계산

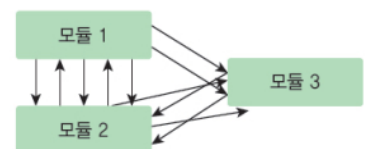
time(NULL); 현재 시간을 초단위로 반환. UNIX시간(1970년 1월 1일 이후의 시간)

모듈화

모듈 내에서 상호작용이 있어야 함. 모듈 사이에는 최소의 상호작용만 존재해야 함. 모듈사이가 복잡하다면 잘못된 것.



(a) 좋은 모듈화



(b) 나쁜 모듈화

9장 함수와 변수

변수의 속성

: 이름, 타입, 크기, 값 + 범위, 생존시간 연결

범위(Scope) : 변수가 사용 가능한 범위, 가시성

생존 시간(lifetime) : 메모리에 존재하는 시간

연결(linkage) : 다른 영역에 있는 변수와의 연결 상태

지역변수

블록안에 선언되는 변수. 블록이 끝나면 소멸됨.

초기화를 안하면 쓰레기값이 들어감.

함수 헤더에 있는 매개변수도 일종의 지역 변수. 이 변수는 호출시 인수 값으로 초기화가 됨.

전역변수

함수 외부에서 선언되는 변수

초기화 안하면 0으로 초기화됨

전역변수와 지역변수가 같은 이름일 때는 지역변수 우선.

생존기간

정적 할당 : 프로그램 실행 시간 동안 계속 유지

자동 할당 : 블록에 들어갈 때 생성, 블록에서 나올 때 소멸

auto

변수를 선언한 위치에서 자동으로 만들어지고 블록을 벗어나게 되며 자동으로 소멸되는 저장 유형을 지정
지역변수는 auto 생략해도 자동으로 auto변수가 됨.

static

지역변수가 **정적변수**가 됨.
블록을 벗어나도 메모리에서 내려오지 않음.

register

CPU안에 레지스터에 변수를 저장함. 접근 속도가 빠름

volatile

하드웨어가 수시로 변수의 값을 변경하는 경우에 사용됨.
캐시하지 않고 **매번 메모리에서 읽도록 함.**

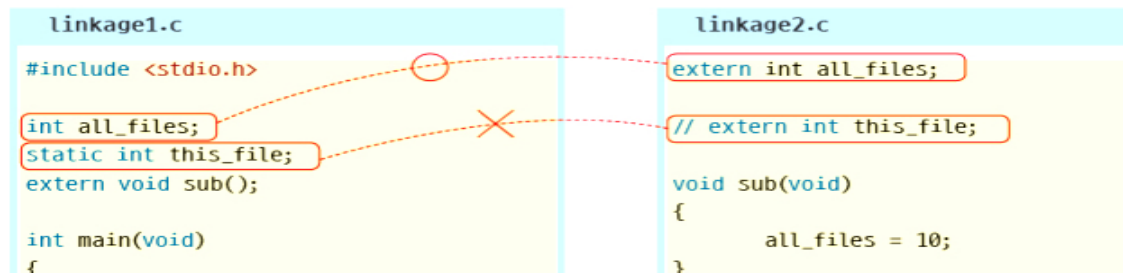
```
void sub() {  
    static int scount = 0;  
    int account = 0;  
    printf("scount = %d\t", scount);  
}
```

```
register int i;  
for(i = 0; i < 100; i++)  
    sum += i;  
  
volatile int io_port; // 하드웨어와 연결된 변수  
  
void wait(void) {  
    io_port = 0;  
    while (io_port != 255)  
        ;  
}
```

연결(linkage)

다른 범위에 속하는 변수들을 서로 연결하는 것
전역 변수만이 연결을 가질 수 있음.

외부 연결



extern int global;

static은 extern할 수 없음 - static은 그 파일 안에서만 쓸 수 있다.

한 파일 안의 블록 안에서 전역 변수에 접근할 때도 쓴다.

```
#include <stdio.h>  
int x = 50;  
  
int main(void)  
{  
    int x = 100;  
    {  
        extern int x;  
        printf("x= %d\n", x); //x=50  
    }  
    return 0;  
}
```

일반적으로는 auto 권장.

변수의 값이 함수가 끝나도 필요하다면 static
많은 함수에서 공유되어야 하는 변수라면 extern

가변 매개변수

매개변수의 개수를 가변적으로 바꿀 수 있다.

```
int sum( int num, ... )
```

```
sum(4, 10 20, 30, 40);
```

순환(Recursion)이란?

자기 자신을 호출!

$$n! = \begin{cases} 1 & n=0 \\ n*(n-1)! & n \geq 1 \end{cases}$$

팩토리얼

```
int factorial(int n)
{
```

```
    if( n <= 1 ) return 1
```

```
    else return n * factorial(n-1);
```

```
}
```

← 순환을 멈추는 부분

← 순환호출을 하는 부분

```
#include <stdarg.h>
```

```
int sum( int, ... );
```

```
int main( void )
```

```
{
```

```
    int answer = sum( 4, 4, 3, 2, 1 );
```

```
    printf( "합은 %d입니다.\n", answer );
```

```
    return( 0 );
```

```
}
```

```
int sum( int num, ... )
```

```
{    int answer = 0;
```

```
    va_list argptr;
```

```
    va_start( argptr, num );
```

```
    for( ; num > 0; num-- )
```

```
        answer += va_arg( argptr, int );
```

```
    va_end( argptr );
```

```
    return( answer );
```

```
}
```

합은 10입니다.

10진수 -> 2진수

```
void print_binary(int x)
```

```
{
```

```
    if (x > 0)
```

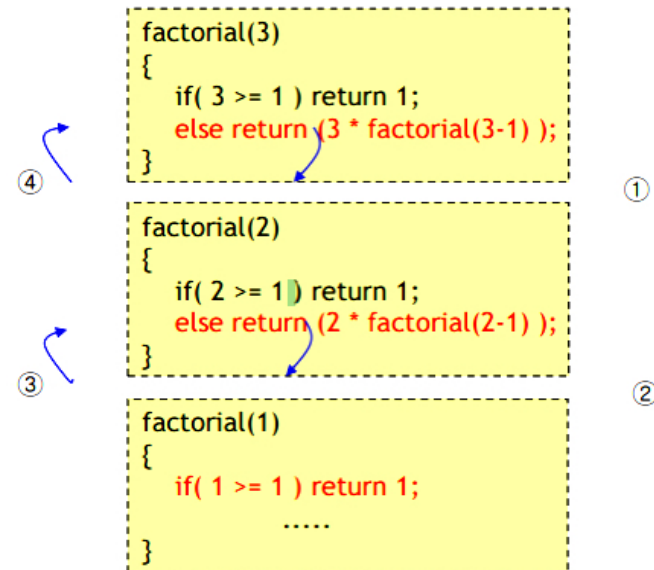
```
    {
```

```
        print_binary(x / 2); // 재귀 호출
```

```
        printf("%d", x % 2); // 나머지를 출력
```

```
    }
```

```
}
```



최대공약수

```
// x는 y보다 커야 한다.
```

```
int gcd(int x, int y)
```

```
{
```

```
    if (y == 0)
```

```
        return x;
```

```
    else
```

```
        return gcd(y, x % y);
```

```
}
```

하노이타워

```
void hanoi_tower(int n, char from, char tmp, char to)
```

```
{
```

```
    if (n == 1)
```

```
        printf("원판 1을 %c에서 %c으로 옮긴다.\n", from, to);
```

```
    else
```

```
    {
```

```
        hanoi_tower(n - 1, from, to, tmp);
```

```
        printf("원판 %d을 %c에서 %c으로 옮긴다.\n", n, from, to);
```

```
        hanoi_tower(n - 1, tmp, from, to);
```

```
    }
```

```
}
```

10장 배열

배열의 특징

메모리의 **연속적인 공간**에 저장. 메모리 상에 붙어 있음.
차례로 접근해서 처리할 수 있다는 점이 장점.

out of index 안나오게 조심!

배열 초기화

중괄호로 감싸면 된다.

```
int scores[5]={10,20,30,40,50}   등등..
```

```
int scores[5]={10,20,30}           //비워놓으면 나머지는 자동으로 0으로 초기화.
```

```
int score[]={10,20,30,40,50}       //배열 크기 비워놓으면 자동으로 크기가 배열 크기만큼으로 할당
```

```
int score[5];                       //초기화 안하면 쓰레기값!
```

```
size = sizeof(scores) / sizeof(scores[0]) //원소 갯수 계산
```

배열의 복사/비교

그냥 대입/비교하면 안되고 원소를 for문으로 하나씩 복사해야 한다.

배열의 인수 전달

```
void modify_array(int a[], int size)
```

배열은 주소를 전달하고, 배열의 원본이 전달된다.

```
void print_array(const int a[], int size)
```

const로 전달하면 배열 값이 변경되지 않음.

정렬

선택정렬 selection sort

순차탐색

배열에서 처음부터 하나씩 찾는 방법

이진탐색

```
int binary_search(int list[], int n, int key)
{
    int low, high, middle;
    low = 0;
    high = n-1;
    while( low <= high ){
        // 아직 숫자들이 남아있으면
        printf("[%d %d]\n", low, high); // 하한과 상한을 출력한다.
        middle = (low + high)/2;         // 중간 위치를 계산한다.
        if( key == list[middle] )        // 일치하면 탐색 성공
            return middle;
        else if( key > list[middle] )    // 중간 원소보다 크다면
            low = middle + 1;           // 새로운 값으로 low 설정
        else
            high = middle - 1;          // 새로운 값으로 high 설정
    }
    return -1;
}
```

2차원 배열

```
int s[3][5] = {
    { 0, 1, 2, 3, 4 }, // 첫 번째 행의 원소들의 초기값
    { 10, 11, 12, 13, 14 }, // 두 번째 행의 원소들의 초기값
    { 20, 21, 22, 23, 24 } // 세 번째 행의 원소들의 초기값
};
```

앞에서 비우는거 가능!

```
int s[ ][5] = {
    0, 1, 2, 3, 4, 10, 11, 12, 13, 14, 20, 21, 22, 23, 24
};
```

2차원 배열 함수로 전달

```
int sum(int scores[YEARS][PRODUCTS])
{
    int y, p;
    int total = 0;

    for (y = 0; y < YEARS; y++)
        for (p = 0; p < PRODUCTS; p++)
            total += scores[y][p];

    return total;
}
```

11장 포인터

주소

```
int main(void)
{
    int i = 10;
    char c = 69;
    float f = 12.3;

    printf("i의 주소: %p\n", &i); // 변수 i의 주소 출력
    printf("c의 주소: %p\n", &c); // 변수 c의 주소 출력
    printf("f의 주소: %p\n", &f); // 변수 f의 주소 출력
    return 0;
}
```

포인터 선언

```
int *p1, *p2, *p3;
```

```
int i = 10; // 정수형 변수 i 선언
int *p; // 포인터 변수 p 선언
p = &i; // 변수 i의 주소가 포인터 p로 대입
```

```
char c = 'A'; // 문자형 변수 c
float f = 36.5; // 실수형 변수 f
double d = 3.141592; // 실수형 변수 d
```

NULL 포인터는 아무것도 가리키지 않음!

```
char *pc = &c; // 문자를 가리키는 포인터 pc
float *pf = &f; // 실수를 가리키는 포인터 pf
double *pd = &d; // 실수를 가리키는 포인터 pd
```

간접 참조 연산자 * :

포인터가 가리키는 값을 reference

```
int i = 10;

int* p;
p = &i;

printf("%d\n", *p);
```

```
int *pi = (int *)10000;
char *pc = (char *)10000;
double *pd = (double *)10000;
```

- & 연산자: 변수의 주소를 반환한다
- * 연산자: 포인터가 가리키는 곳의 내용을 반환한다.

포기화를 하지 않은 포인터를 사용하면 안 됨. NULL로라도 초기화를 해 줘야 한다.

변수와 포인터의 타입은 같아야 한다.

----->

```
#include <stdio.h>

int main(void)
{
    int i;
    double *pd;

    pd = &i;           // 오류!
    *pd = 36.5;

    return 0;
}
```

포인터 연산

포인터 타입	++연산후 증가되는값
char	1
short	2
int	4
float	4
double	8

```
char *pc;
int *pi;
double *pd;

pc = (char *)10000;
pi = (int *)10000;
pd = (double *)10000;
printf(" pc=%u, pc+1=%u, pc+2= %u\n", pc, pc + 1, pc + 2);
printf(" pi=%u, pi+1=%u, pi+2= %u\n", pi, pi + 1, pi + 2);
printf(" pd=%u, pd+1=%u, pd+2= %u\n", pd, pd + 1, pd + 2);
```

pc=10000, pc+1=10001, pc+2= 10002
 pi=10000, pi+1=10004, pi+2= 10008
 pd=10000, pd+1=10008, pd+2= 10016

수식	의미
v = *p++	p가 가리키는 값을 v에 대입한 후에 p를 증가한다.
v = (*p)++	p가 가리키는 값을 v에 대입한 후에 가리키는 값을 증가한다.
v = *++p	p를 증가시킨 후에 p가 가리키는 값을 v에 대입한다.
v = ++*p	p가 가리키는 값을 가져온 후에 그 값을 증가하여 v에 대입한다.

```
int i = 10;
int *pi = &i;

printf("i = %d, pi = %p\n", i, pi);

(*pi)++;
printf("i = %d, pi = %p\n", i, pi);

*pi++;
printf("i = %d, pi = %p\n", i, pi);
return 0;
```

i = 10, pi = 000000FFEB9CFF974
 i = 11, pi = 000000FFEB9CFF974
 i = 11, pi = 000000FFEB9CFF978

포인터 형변환

```
double* pd = &f;
int* pi;
pi = (int*)pd;
```

```
int data = 0x0A0B0C0D;
char *pc;
int i;
pc = (char *)&data;
for (i = 0; i < 4; i++)
    printf("(pc + %d) = %02X \n", i, *(pc + i));
```

*(pc + 0) = 0D 리틀엔디언이라! 실제로는 0D 0C 0B 0A 이런식으로 저장되어 있음.

*(pc + 1) = 0C

*(pc + 2) = 0B

*(pc + 3) = 0A

인수 전달 방법

값에 의한 호출 val by value

복사본이 전달. C언어에서 기본적인 방법

참조에 의한 호출 call by reference

원본이 호출. 포인터를 이용하여 흉내 낼 수 있다.

```
#include <stdio.h>
void swap(int x, int y);
int main(void)
{
    int a = 100, b = 200;
    printf("a=%d b=%d\n", a, b);

    swap(a, b);

    printf("a=%d b=%d\n", a, b);
    return 0;
}
```

```
void swap(int x, int y)
{
    int tmp;
    printf("x=%d y=%d\n", x, y);

    tmp = x;
    x = y;
    y = tmp;

    printf("x=%d y=%d\n", x, y);
}
```

a=100 b=200
x=100 y=200
x=200 y=100
a=100 b=200

```
#include <stdio.h>
void swap(int x, int y);
int main(void)
{
    int a = 100, b = 200;
    printf("a=%d b=%d\n", a, b);

    swap(&a, &b);

    printf("a=%d b=%d\n", a, b);
    return 0;
}
```

```
void swap(int *px, int *py)
{
    int tmp;

    tmp = *px;
    *px = *py;
    *py = tmp;
}
```

a=100 b=200
a=200 b=100

함수가 포인터를 통하여 값을 변경할 수 없게 하려면?

```
void sub(const int *p)
{
    *p = 0; // 오류!!
}
```

지역 변수를 반환하면 안 된다. 메모리에서 날아가기 때문.

```
int *add(int x, int y)
{
    int result;
    result = x + y;
    return &result;
}
```

포인터와 배열

배열 이름은 포인터이다.

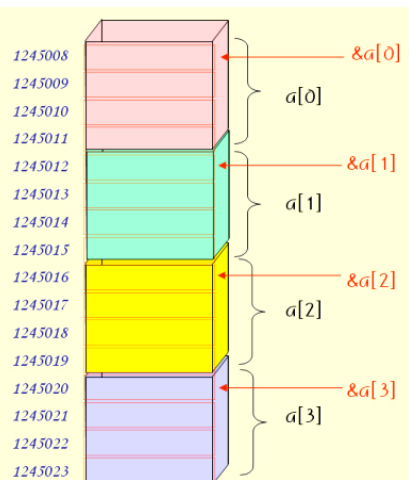
```
// 포인터와 배열의 관계
#include <stdio.h>

int main(void)
{
    int a[] = { 10, 20, 30, 40, 50 };

    printf("&a[0] = %u\n", &a[0]);
    printf("&a[1] = %u\n", &a[1]);
    printf("&a[2] = %u\n", &a[2]);

    printf("a = %u\n", a);

    return 0;
}
```



&a[0] = 1245008
&a[1] = 1245012
&a[2] = 1245016
a = 1245008

```
int a[] = { 10, 20, 30, 40, 50 };
printf("a = %u\n", a);
printf("a + 1 = %u\n", a + 1);
printf("*a = %d\n", *a);
printf("*(a+1) = %d\n", *(a + 1));
return 0;
```

a = 1245008
a + 1 = 1245012
*a = 10
*(a+1) = 20

포인터를 배열처럼 사용

----->

a[0]=10 a[1]=20 a[2]=30
p[0]=10 p[1]=20 p[2]=30
a[0]=60 a[1]=70 a[2]=80
p[0]=60 p[1]=70 p[2]=80

```
int a[] = { 10, 20, 30, 40, 50 };
int *p;
```

구현
있다.

```
p = a;
printf("a[0]=%d a[1]=%d a[2]=%d \n", a[0], a[1], a[2]);
printf("p[0]=%d p[1]=%d p[2]=%d \n\n", p[0], p[1], p[2]);
```

```
p[0] = 60;
p[1] = 70;
p[2] = 80;
```

```
printf("a[0]=%d a[1]=%d a[2]=%d \n", a[0], a[1], a[2]);
printf("p[0]=%d p[1]=%d p[2]=%d \n", p[0], p[1], p[2]);
return 0;
```

일반 매개 변수 vs 배열 매개 변수

```
// 매개 변수 x에 기억 장소가 할당
void sub(int x)
{
    ...
}
```

```
// b에 기억 장소가 할당되지 않는다.
void sub( int b[] )
{
    ...
}
```

Why? -> 배열을 함수로 복사하려면 많은 시간 소모되므로 배열은 주소만을 전달한다.

```
void sub(int b[], int n);
int main(void)
{
    int a[3] = { 1,2,3 };
    printf("%d %d %d\n", a[0], a[1], a[2]);
    sub(a, 3);
    printf("%d %d %d\n", a[0], a[1], a[2]);
    return 0;
}

void sub(int b[], int n)
{
    b[0] = 4;
    b[1] = 5;
    b[2] = 6;
}
```

결과 : 1 2 3
4 5 6

이 두가지는 같다!

```
// 포인터 매개 변수
void sub(int *b, int size)
{
    b[0] = 4;
    b[1] = 5;
    b[2] = 6;
}
```

배열의 이름과 포인터
는 근본적으로 같다.

배열 표기법을 사용
하여 요소에 접근

```
// 포인터 매개 변수
void sub(int *b, int size)
{
    *b = 4;
    *(b+1) = 5;
    *(b+2) = 6;
}
```

포인터 표기법을 사
용하여 요소에 접근

포인터를 사용하는 것이 인덱스 표기법보다 빠르다!

포인터 사용의 장점

연결 리스트나 이진 트리 등의 향상된 자료 구조를 만들 수 있다.

참조에 의한 호출로 포인터를 매개 변수로 이용하여 함수 외부의 변수의 값을 변경할 수 있다.

메모리 매핑 하드웨어에서 메모리를 제어할 수 있다.

동적 메모리 할당 가능.